



ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
ΣΧΟΛΗ ΠΛΗΡΟΦΟΡΙΚΗΣ & ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ
ΠΑΝΕΠΙΣΤΗΜΙΟ ΙΩΑΝΝΙΝΩΝ



Αλέξανδρος Μπανταλούκας-Αρτζμάντ

Αρχιτεκτονική Υπολογιστών Εργαστήριο

1

Αρχιτεκτονική Υπολογιστών Εργαστήριο

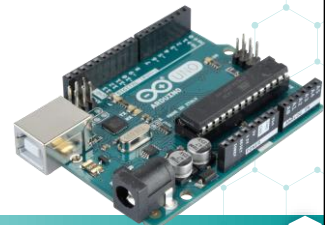
Διδάσκων:

Αλέξανδρος Μπανταλούκας-Αρτζμάντ MSc, PhD

k.arjmand@uoi.gr



Αλέξανδρος Μπανταλούκας-Αρτζμάντ



Αρχιτεκτονική Υπολογιστών Εργαστήριο

2

3^η Εργασία Εξαμήνου

MIPS σε VHDL

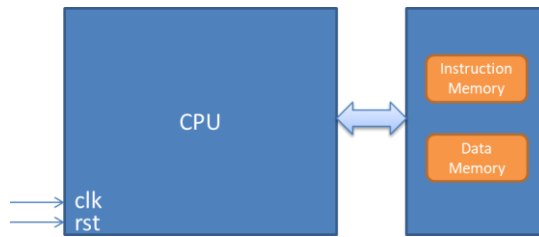
Στόχος της εργασίας είναι ο σχεδιασμός, η υλοποίηση σε VHDL και η προσομοίωση ενός τμήματος ενός απλού μικροεπεξεργαστή 32-bit.

Η σχεδίαση θα περιλαμβάνει ένα υποσύνολο εντολών του μικροεπεξεργαστή **MIPS**, καθώς και ορισμένες υπομονάδες του, απαραίτητες για τη λειτουργία του.

Αρχιτεκτονική Συστήματος

Η αρχιτεκτονική του συστήματος αποτελείται από τα εξής:

1. **Μνήμη συστήματος:** περιλαμβάνει τη μνήμη προγράμματος και τη μνήμη δεδομένων. Η μνήμη προγράμματος είναι οργανωμένη σε 256 λέξεις των 32 bit, με διευθύνσεις που ξεκινούν από το 0. Το εύρος των διευθύνσεων περιορίζεται σε 1 Byte
2. **Μικροεπεξεργαστής (CPU) 32-bit:** είναι υπεύθυνος για την εκτέλεση των εντολών και τη διαχείριση των λειτουργιών της μνήμης



Το κύκλωμα MIPS (1)

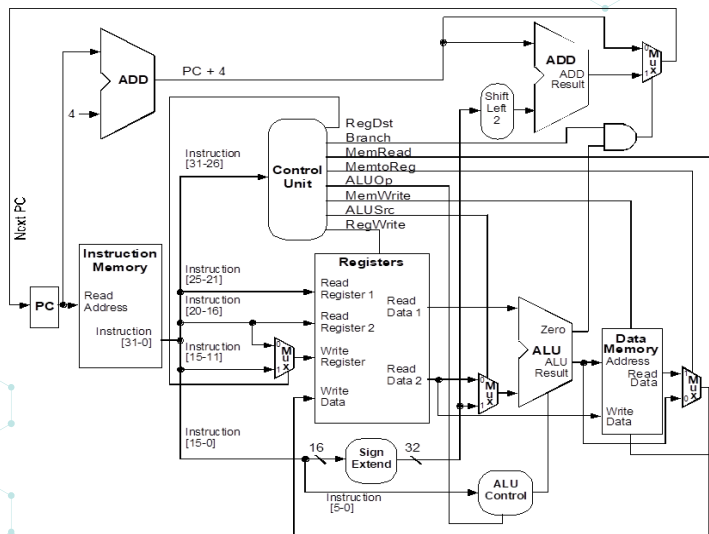
Το κύκλωμα αποτελείται από βασικά μέρη, καθένα από τα οποία επιτελεί συγκεκριμένες λειτουργίες που διευκρινίζονται παρακάτω:

1. **Αριθμητική και Λογική Μονάδα (ALU):** εκτελεί όλες τις απαραίτητες αριθμητικές και λογικές λειτουργίες που απαιτούνται για την εφαρμογή του επιλεγμένου συνόλου εντολών. Επίσης, διαχειρίζεται τις συνθήκες διακλάδωσης (Branches)
2. **Αρχείο Καταχωρητών:** περιλαμβάνει 32 καταχωρητές με δύο θύρες ανάγνωσης και μία θύρα εγγραφής, διευκολύνοντας την αποθήκευση και την ανάκτηση δεδομένων
3. **Καταχωρητής Εντολών (IR):** αποθηκεύει την τρέχουσα εντολή που έχει προσκομιστεί από τη μνήμη

Το κύκλωμα MIPS (2)

4. **Απαριθμητής Προγράμματος (PC):** περιέχει τη διεύθυνση μνήμης της επόμενης εντολής που πρόκειται να εκτελεστεί, διασφαλίζοντας τη σωστή ροή του προγράμματος
5. **Μηχανισμός Ελέγχου (Control):** ρυθμίζει και κατευθύνει τη λειτουργία του DataPath και της μνήμης. Επιπλέον, συνεργάζεται με την μονάδα ελέγχου της **ALU**, χρησιμοποιώντας σήματα από τον **Control** και το **DataPath** για την επιτυχή εκτέλεση εντολών
6. **Μηχανισμός ALU Control:** υπεύθυνος για την παραγωγή των σημάτων ελέγχου που κατευθύνουν την ALU
7. **Μηχανισμός Επέκτασης Προσήμανσης:** μετατρέπει σήματα 16-bit σε 32-bit, διευκολύνοντας τις πράξεις όπου απαιτείται η προσαρμογή μήκους δεδομένων

Διάγραμμα Κυκλώματος MIPS



Αλέξανδρος Μπανταλούκας-Αρτζμάντ

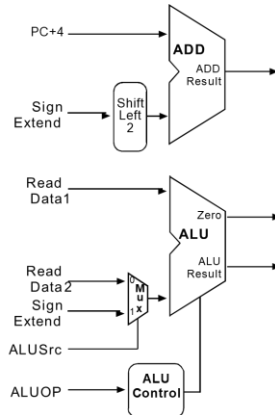
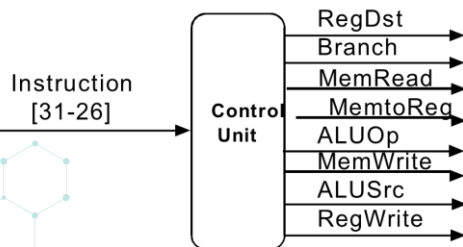
Αρχιτεκτονική Υπολογιστών Εργαστήριο

7

Υλοποίηση Υπομονάδων MIPS

Στόχος της Εργασίας είναι η υλοποίηση δύο βασικών υπομονάδων του MIPS:

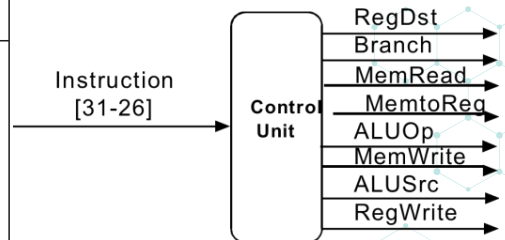
1. Μονάδα Ελέγχου (**Control Unit**)
2. Αριθμητική και Λογική Μονάδας (**ALU**)



Υπομονάδα 1 - Μονάδα Ελέγχου

Αρχικά, θα σχεδιάσουμε τη Μονάδα Ελέγχου ώστε να παράγει τα απαραίτητα σήματα για την εκτέλεση των ακόλουθων εντολών.

Name	Op-code					
	Op5	Op4	Op3	Op2	Op1	Op0
R-format	0	0	0	0	0	0
lw	1	0	0	0	1	1
sw	1	0	1	0	1	1
beq	0	0	0	1	0	0
j	0	0	0	0	1	0

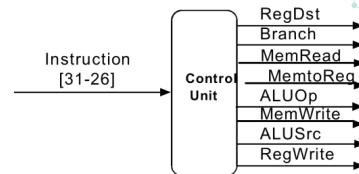


Υπομονάδα 1 - Μονάδα Ελέγχου

Το πεδίο λειτουργίας (**instruction**) θα πρέπει να αποκωδικοποιηθεί για να παραχθεί το απαιτούμενο σήμα ελέγχου που κατευθύνει τη λειτουργία της ALU.

Για την υλοποίηση, θα χρησιμοποιηθεί ξεχωριστός αποκωδικοποιητής τόσο για τα κύρια σήματα ελέγχου όσο και για τον έλεγχο της ALU.

Name	Op-code					
	Op5	Op4	Op3	Op2	Op1	Op0
R-format	0	0	0	0	0	0
lw	1	0	0	0	1	1
sw	1	0	1	0	1	1
beq	0	0	0	1	0	0
j	0	0	0	0	1	0



Υπομονάδα 1 - Μονάδα Ελέγχου

Τα σήματα που απαιτούνται για τον έλεγχο της διαδρομής δεδομένων είναι τα εξής:

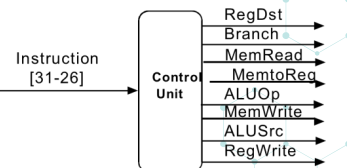
- **Jump:** ορίζεται σε **1** για εντολές άλματος
- **Branch:** ορίζεται σε **1** για εντολές διακλάδωσης
- **MemtoReg:** ορίζεται σε **1** για εντολές φόρτωσης
- **ALUSrc:** ορίζεται σε **0** για εντολές τύπου R (R-type) και **1** για εντολές με άμεσα δεδομένα (στο *beq* ορίζεται σε 0)
- **RegDst:** ορίζεται σε **1** για εντολές τύπου R και **0** για άμεσες εντολές
- **MemRead:** ορίζεται σε **1** για εντολές φόρτωσης
- **MemWrite:** ορίζεται σε **1** για εντολές αποθήκευσης
- **RegWrite:** ορίζεται σε **1** για οποιαδήποτε εντολή εγγραφής σε έναν καταχωρητή
- **ALUOp (k bit):** κωδικοποιεί τις εντολές ALU, εκτός από τις τύπου R, οι οποίες καθορίζονται από το πεδίο λειτουργίας

Υπομονάδα 1 - Μονάδα Ελέγχου

Για τις εντολές που υποστηρίζονται, το σήμα **ALUOp** κωδικοποιείται σε 2 bit, σύμφωνα με τον παρακάτω πίνακα:

ALUOp [1]	ALUOp [0]	Instruction
0	0	memory operations (<i>load</i> , <i>store</i>)
0	1	<i>beq</i>
1	0	r-type operations

Αυτή η κωδικοποίηση επιτρέπει τη διαφοροποίηση των λειτουργιών της ALU, με βάση τα σήματα ελέγχου που παράγονται από τη **Μονάδα Ελέγχου (Control Unit)**.

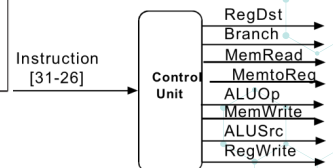


- **ALUOp[1:0] = 00:** Εντολές μνήμης (φόρτωση και αποθήκευση)
- **ALUOp[1:0] = 01:** Εντολή διακλάδωσης (*beq*)
- **ALUOp[1:0] = 10:** Εντολές τύπου R (αριθμητική και λογική επεξεργασία)

Υπομονάδα 1 - Μονάδα Ελέγχου - Σήματα Ελέγχου (1)

Ο παρακάτω πίνακας παρουσιάζει τις απαιτούμενες τιμές για τα σήματα ελέγχου της Μονάδας Ελέγχου, ανάλογα με τον κωδικό λειτουργίας (**Op-code**) της κάθε εντολής:

Instruction	Op-code	RegDst	ALUSrc	MemtoReg	Reg Write
r-type	0 0 0 0 0 0	1	0	0	1
lw	1 0 0 0 1 1	0	1	1	1
sw	1 0 1 0 1 1	x	1	x	0
beq	0 0 0 1 0 0	x	0	x	0
j	0 0 0 0 1 0	x	x	x	0



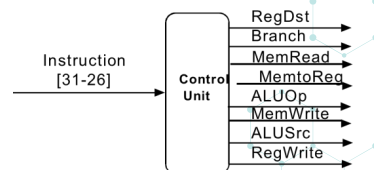
- **RegDst:** καθορίζει αν ο προορισμός εγγραφής είναι ένας καταχωρητής τύπου R ή ένας άλλος (0 ή 1)
- **ALUSrc:** ορίζει αν η ALU θα χρησιμοποιήσει άμεση τιμή ή δεδομένα από δεύτερο καταχωρητή
- **MemtoReg:** ενεργοποιείται για την επιστροφή δεδομένων από τη μνήμη σε έναν καταχωρητή
- **RegWrite:** ενεργοποιεί την εγγραφή σε καταχωρητή

Υπομονάδα 1 - Μονάδα Ελέγχου - Σήματα Ελέγχου (2)

Ο παρακάτω πίνακας συνοψίζει τις τιμές των σημάτων ελέγχου που απαιτούνται για της σωστή εκτέλεση κάθε εντολής, με βάση τον κωδικό λειτουργίας (**Op-code**):

Instruction	Op-code	Mem Read	Mem Write	Branch	ALUOp[1:0]	Jump
r-type	0 0 0 0 0 0	0	0	0	1 0	0
lw	1 0 0 0 1 1	1	0	0	0 0	0
sw	1 0 1 0 1 1	0	1	0	0 0	0
beq	0 0 0 1 0 0	0	0	1	0 1	0
j	0 0 0 0 1 0	0	0	0	x x	1

Η **Μονάδα Ελέγχου** χρησιμοποιεί τον κωδικό λειτουργίας (**Instruction [31-26]**) για την παραγωγή των αντίστοιχων σημάτων ελέγχου. Τα σήματα αυτά συντονίζουν τις λειτουργίες της ALU, της μνήμης και των άλλων υποσυστημάτων.



Αλέξανδρος Μπανταλούκας-Αρτζμάντ

Αρχιτεκτονική Υπολογιστών Εργαστήριο

14

Instruction [31-26]: αναφέρονται στα πιο σημαντικά 6 bits του **32-bit instruction word** στην αρχιτεκτονική **MIPS**. Αυτά τα bits συνιστούν το **opcode** (operation code) και καθορίζουν τη βασική λειτουργία της εντολής που θα εκτελεστεί από τον επεξεργαστή, ενεργοποιώντας τα κατάλληλα σήματα ελέγχου, όπως:

- **MemRead:** ενεργοποιείται για φόρτωση δεδομένων από τη μνήμη (load)
- **MemWrite:** ενεργοποιείται για αποθήκευση δεδομένων στη μνήμη (store)
- **Branch:** ενεργοποιείται για εντολές διακλάδωσης (beq)
- **ALUOp[1:0]:** καθορίζει τη λειτουργία της ALU για κάθε τύπο εντολής (π.χ., R-type, I-type)
- **Jump:** ενεργοποιείται για εντολές άλματος (j)

Υπομονάδα 1 - Μονάδα Ελέγχου - Βήματα Υλοποίησης

- ✓ Χρησιμοποιήστε τον παρεχόμενο κώδικα βάσης και τους προηγούμενους πίνακες για να προσθέσετε τις κατάλληλες εντολές που απαιτούνται για την υλοποίηση της Μονάδας Ελέγχου
- ✓ Δημιουργήστε ένα νέο αρχείο VHDL και γράψτε τον κώδικά σας. Ελέγξτε την ορθότητα του κώδικα μέσω του εργαλείου μεταγλώττισης
- ✓ Μετατρέψτε τον κώδικα σε σχηματικό για χρήση σε Block Diagram
- ✓ Δημιουργήστε ένα σχηματικό αρχείο και προσθέστε τις κατάλληλες εισόδους και εξόδους στο κύκλωμα
- ✓ Επαληθεύστε τη λειτουργία του κυκλώματος προσθέτοντας όλα τα απαιτούμενα σήματα εισόδου και παρακολουθώντας τις εξόδους
- ✓ Δημιουργήστε ένα αρχείο κυματομορφών, προσθέστε κατάλληλες τιμές εισόδου και επαληθεύστε τη σωστή λειτουργία του κυκλώματος
- ✓ Δείξτε την ακρίβεια του σχεδιασμού σας χρησιμοποιώντας διαφορετικούς συνδυασμούς τιμών εισόδου

Υπομονάδα 1 - Μονάδα Ελέγχου - Κώδικας Βάσης

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_signed.all;

entity control_unit is
port(
    Opcode : in _____;
    RegDst : out std_logic;
    ALUSrc : out std_logic;
    MemtoReg : out std_logic;
    RegWrite : out std_logic;
    MemRead : out std_logic;
    MemWrite : out std_logic;
    Branch : out std_logic;
    ALUOp : out _____;
    clock, reset : in std_logic
);
end control_unit;
```

```
architecture behavior of control_unit is
    signal R_format, Lw, Sw, Beq, Ji : std_logic;
begin
    -- Code to generate control signals using opcode
    -- bits
    _____ <= _____ when _____ else _____;
    _____ <= _____ when _____ else _____;
    _____ <= _____ when _____ else _____;
    _____ <= _____ when _____ else _____;
    _____ <= _____ when _____ else _____;

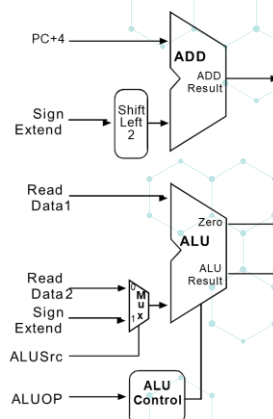
    RegDst <= _____;
    ALUSrc <= _____;
    MemtoReg <= _____;
    RegWrite <= _____;
    MemRead <= _____;
    MemWrite <= _____;
    Branch <= _____;

    ALUOp(_____) <= _____;
    ALUOp(_____) <= _____;
end behavior;
```

Υπομονάδα 2 - Η ALU

Οι είσοδοι στη μονάδα ελέγχου της **ALU** είναι τα σήματα ελέγχου **ALUOp** και το πεδίο λειτουργίας (**funct**) των 6 bit:

- Το πεδίο **funct** καθορίζει τις λειτουργίες της ALU για τις λειτουργίες τύπου **R**, εξειδικεύοντας περαιτέρω τη λειτουργικότητα
- Τα σήματα **ALUOp** χρησιμοποιούνται για τον καθορισμό των λειτουργιών της ALU για άλλους τύπους εντολών, όπως η φόρτωση (load), αποθήκευση (store) και διακλαδώσεις (branch)



Υπομονάδα 2 - Η ALU - Σήματα Ελέγχου (1)

Ο παρακάτω πίνακας παρουσιάζει τις διαφορετικές τιμές που πρέπει να αποδοθούν στα σήματα **ALUOp** και στο πεδίο **funct** για κάθε εντολή.

Instruction	ALUOp	funct	ALU operation	ALU control input
lw	0 0	x x x x x x	add	0 1 0
sw	0 0	x x x x x x	add	0 1 0
beq	0 1	x x x x x x	subtract	1 1 0
add	1 0	1 0 0 0 0 0	add	0 1 0
sub	1 0	1 0 0 0 1 0	subtract	1 1 0
and	1 0	1 0 0 1 0 0	AND	0 0 0
or	1 0	1 0 0 1 0 1	OR	0 0 1
slt	1 0	1 0 1 0 1 0	set on less than	1 1 1

- **ALUOp:** καθορίζει τη γενική κατηγορία της λειτουργίας (π.χ. εντολές μνήμης, διακλάδωσης ή αριθμητικές/λογικές)
- **funct:** όπου απαιτείται, εξειδικεύει τη λειτουργία για τις εντολές τύπου **R**
- **ALU control input:** η τιμή του σήματος που καθοδηγεί την ALU στη σωστή λειτουργία

Υπομονάδα 2 - Η ALU - Σήματα Ελέγχου (2)

Στόχος της εργασίας για τη δεύτερη υπομονάδα είναι να συμπληρώσετε τον παρεχόμενο κώδικα βάσης, με βάση τον παρακάτω πίνακα, ώστε να υλοποιηθούν οι αντίστοιχες λειτουργίες της **ALU**.

Παρουσιάστε αρκετά παραδείγματα χρησιμοποιώντας διαφορετικές τιμές εισόδου. Οι κυματομορφές προσομοίωσης που θα δημιουργηθούν θα πρέπει να επιβεβαιώνουν τη σωστή λειτουργία του κυκλώματος.

Instruction opcode	ALUOp	Instruction Operation	Funct Field	Desired ALU Action	ALU Control Input
lw	00	Load word	XXXXXX	Add	010
sw	00	Store word	XXXXXX	Add	010
beq	01	Branch equal	XXXXXX	Subtract	110
R-type	10	Add (+)	100000	Add	010
R-type	10	Subtract (-)	100010	Subtract	110
R-type	10	AND	100100	And	000
R-type	10	OR	100101	Or	001
R-type	10	XOR	101001	Exclusive Or	011
R-type	10	Set on less than	101010	Set on Less Than	111

Υπομονάδα 2 - Η ALU - Βήματα Υλοποίησης

- ✓ Χρησιμοποιήστε τον παρεχόμενο κώδικα βάσης και τους προηγούμενους πίνακες για να προσθέσετε τις κατάλληλες εντολές που απαιτούνται για την υλοποίηση της **ALU**
- ✓ Δημιουργήστε ένα νέο αρχείο VHDL και γράψτε τον κώδικά σας. Ελέγξτε την ορθότητα του κώδικα μέσω του εργαλείου μεταγλώττισης
- ✓ Μετατρέψτε τον κώδικα σε σχηματικό για χρήση σε Block Diagram
- ✓ Δημιουργήστε ένα σχηματικό αρχείο και προσθέστε τις κατάλληλες εισόδους και εξόδους στο κύκλωμα
- ✓ Επαληθεύστε τη λειτουργία του κυκλώματος προσθέτοντας όλα τα απαιτούμενα σήματα εισόδου και παρακολουθώντας τις εξόδους
- ✓ Δημιουργήστε ένα αρχείο κυματομορφών, προσθέστε κατάλληλες τιμές εισόδου και επαληθεύστε τη σωστή λειτουργία του κυκλώματος
- ✓ Δείξτε την ακρίβεια του σχεδιασμού σας χρησιμοποιώντας διαφορετικούς συνδυασμούς τιμών εισόδου

Υπομονάδα 2 - Η ALU - Κώδικας Βάσης (1)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_signed.all;

entity ALU is
port(
    Read_data_1 : in _____;
    Read_data_2 : in _____;
    Sign_extend : in std_logic_vector(31 downto 0);
    Function_opcode : in std_logic_vector(5 downto 0);
    ALUOp : in _____;
    ALUSrc : in std_logic;
    Zero : out std_logic;
    ALU_Result : out _____;
    Add_Result : out std_logic_vector(7 downto 0);
    PC_plus_4 : in std_logic_vector(7 downto 0);
    clock, reset : in std_logic;
);
end ALU;
```

Αλέξανδρος Μπανταλούκας-Αρτζμάντ

Αρχιτεκτονική Υπολογιστών Εργαστήριο

21

1. Τιμές εισόδων:

- **Sign_extend**: ένα std_logic_vector που χρησιμοποιείται για την επέκταση πρόσημου:
 - **Χρήση**: επιτρέπει τη μετατροπή αριθμών μικρότερης ακρίβειας σε μεγαλύτερη, διατηρώντας το πρόσημο. Χρησιμοποιείται για branch εντολές ή αριθμητικές πράξεις
 - **Αρχική τιμή**: πρέπει να δοθεί ένας αριθμός 32-bit που επεκτείνει τον αρχικό αριθμό (π.χ., για υπολογισμούς διευθύνσεων branch)
- **ALUSrc**: είναι ένας std_logic που καθορίζει ποια είσοδος θα χρησιμοποιηθεί από την ALU:
 - **ALUSrc = '0'**: η ALU λαμβάνει το **Read_data_2** ως είσοδο
 - **ALUSrc = '1'**: η ALU λαμβάνει το **Sign_extend** ως είσοδο
 - **Τιμή εισόδου**: εξαρτάται από την επιθυμητή λειτουργία. Για παράδειγμα, στις branch εντολές, **ALUSrc = '1'**
- **PC_plus_4**: ένα std_logic_vector που περιέχει την τιμή του προγράμματος καταχωρητή (Program Counter) αυξημένη κατά 4:
 - **Χρήση**: συνδέεται με τον υπολογισμό διευθύνσεων branch ή την επόμενη διεύθυνση εντολής
 - **Τιμή**: πρέπει να αντιστοιχεί στην αυξημένη τιμή του Program Counter

Υπομονάδα 2 - Η ALU - Κώδικας Βάσης (2)

```
architecture behavior of ALU is
    signal Ainput, Binput : std_logic_vector(31 downto 0);
    signal ALU_output_mux : std_logic_vector(31 downto 0);
    signal Branch_Add : std_logic_vector(7 downto 0);
    signal ALU_ctl : std_logic_vector(2 downto 0);
begin
    Ainput <= Read_data_1; -- ALU input mux
    Binput <= Read_data_2

    when (ALUSrc = '0')
    else Sign_extend(31 downto 0);

    -- Generate ALU control bits
    ALU_ctl(0) <= (Function_opcode(0) OR Function_opcode(3)) AND ALUOp(1);
    ALU_ctl(1) <= (NOT Function_opcode(2)) OR (NOT ALUOp(1));
    ALU_ctl(2) <= (Function_opcode(1) AND ALUOp(1)) OR ALUOp(0);

    -- Generate Zero Flag
    Zero <= '1'
    when (ALU_output_mux(31 downto 0) = X"00000000")
    else '0';

    ...
end behavior;
```

Αλέξανδρος Μπανταλούκας-Αρτζμάντ

Αρχιτεκτονική Υπολογιστών Εργαστήριο

22

2. Τιμές εξόδων και λειτουργίες:

- **Zero:** ένα std_logic που λαμβάνει τιμή '1' όταν το αποτέλεσμα της ALU είναι 0, διαφορετικά λαμβάνει τιμή '0':
 - **Χρήση:** κυρίως για branch εντολές (**beq**, **bne**), ώστε να ελέγχεται αν οι τιμές εισόδου είναι ίσες και να επιτρέπεται η δυναμική αλλαγή ροής του προγράμματος
- **ALU_Ctrl:** ένας μηχανισμός που συνδυάζει τα σήματα **ALUOp** και **Funct**:
 - **Χρήση:** παράγει το σήμα ελέγχου που κατευθύνει την ALU στην εκτέλεση μιας λειτουργίας (διακλαδωση, μαθηματική ή λογική πράξη)
- **ALU_Result:** το αποτέλεσμα της πράξης που εκτελείται από την ALU:
 - **Χρήση:** παράγεται βάσει του **ALU_ctl**, το οποίο καθορίζει την πράξη (π.χ., πρόσθεση, AND, OR)
- **Add_Result:** υπολογίζεται από την πρόσθεση του **PC_plus_4** με το **Sign_extend**:
 - **Χρήση:** χρησιμοποιείται στον υπολογισμό διευθύνσεων branch

Υπομονάδα 2 - Η ALU - Κώδικας Βάσης (3)

```

...

-- ALU output for SLT (Set Less Than)
ALU_result <= X"00000000" & B"000" & ALU_output_mux(31)
when ALU_ctl = "111"
else ALU_output_mux(31 downto 0);

-- Adder to compute Branch Address
Branch_Add <= PC_plus_4(7 downto 2) + Sign_extend(7 downto 0);
Add_result <= Branch_Add(7 downto 0);

process (ALU_ctl, Ainput, Binput)
begin
    -- Select ALU operation
    case ALU_ctl is
        when _____ => _____ <= _____;
        when _____ => _____ <= _____;
        when _____ => _____ <= _____;
        when _____ => _____ <= _____;
        when _____ => _____ <= _____;
        when _____ => _____ <= _____;
        when _____ => _____ <= _____;
        when others => _____ <= _____;
    end case;
end process;
end behavior;

```

Αλέξανδρος Μπανταλούκας-Αρτζμάντ

Αρχιτεκτονική Υπολογιστών Εργαστήριο

23

3. Κυματομορφές:

- **Zero Output:** όταν το **ALU_output_mux = 0**, το **Zero** θα έχει τιμή '1'. Σε διαφορετική περίπτωση, **Zero = '0'**
- **ALU_Result:** η έξοδος **ALU_Result** αλλάζει ανάλογα με τη λειτουργία που ορίζεται από το **ALU_ctl**
- **Branch Addressing:** υπολογίζεται από το άθροισμα των τιμών **PC_plus_4** και **Sign_extend**, εξασφαλίζοντας την ακρίβεια στις branch διευθύνσεις

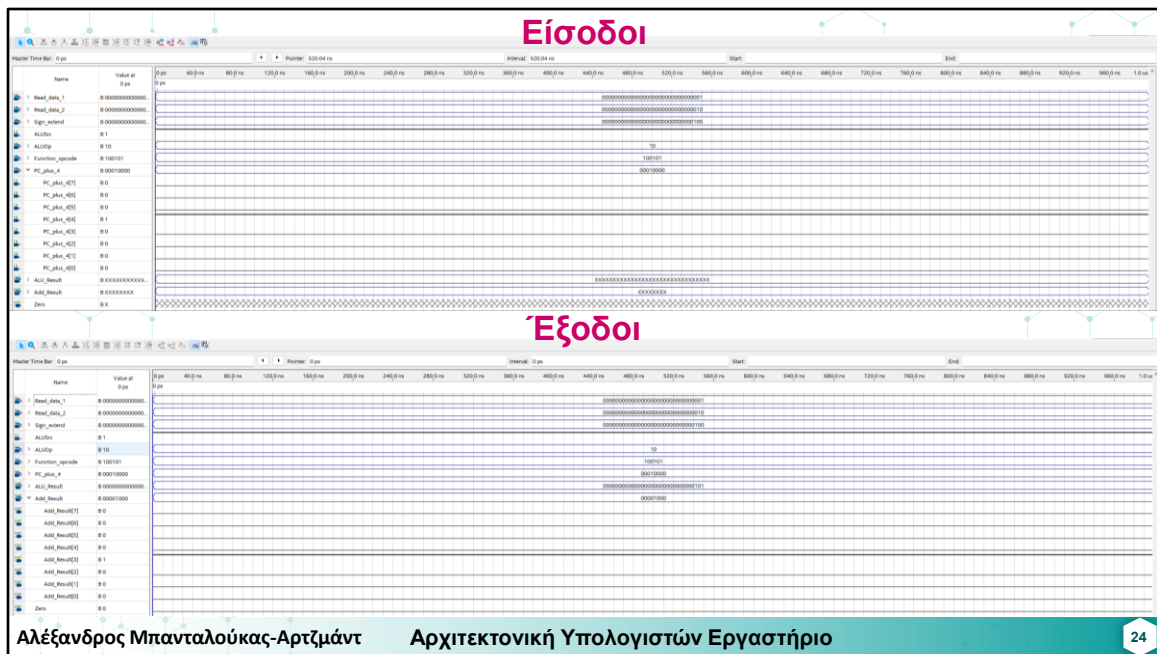
mux: αναφέρεται στον **multiplexer** (πολυπλέκτη), ένα κύκλωμα που, βάσει των σημάτων ελέγχου, επιλέγει μία από πολλές διαθέσιμες εισόδους και την προωθεί ως την έξοδο.

X"00000000": αναπαριστά δεκαεξαδική αναπαράσταση επτά μηδενικών ($7 \times 4 = 28$ bits), τα οποία καλύπτουν τα 28 πιο σημαντικά bits (MSB) του αποτελέσματος:

- **Χρήση:** στην εντολή **SLT**, μόνο το πιο σημαντικό bit της εξόδου **ALU_output_mux(31)** καθορίζει αν ο πρώτος τελεστής (**Ainput**) είναι μικρότερος από τον δεύτερο (**Binput**). Τα υπόλοιπα 28 bits γεμίζονται με μηδενικά για να διατηρηθεί η συμβατότητα με το μήκος των 32 bits της εξόδου

B"000": προσθέτει 3 επιπλέον μηδενικά μετά τα 28 πιο σημαντικά bits (**X"00000000"**), διαμορφώνοντας τα bits 28–30 της εξόδου:

- **Χρήση:** συμπληρώνει τη μορφοποίηση του αποτελέσματος, διασφαλίζοντας ότι το αποτέλεσμα της εντολής **SLT** είναι ένα πλήρως ευθυγραμμισμένο 32-bit αποτέλεσμα, όπου μόνο το τελευταίο bit καθορίζει την κατάσταση της σύγκρισης



Παράδειγμα εφαρμογής:

1. Είσοδοι:

▪ Read_data:

- **Read_data_1:** είναι 00000000000000000000000000000001, δηλαδή 1 σε δυαδική μορφή
- **Read_data_2:** είναι 00000000000000000000000000000010, δηλαδή 2 σε δυαδική μορφή

▪ Sign_extend: 00000000000000000000000000000100, δηλαδή 4

▪ ALUSrc: η τιμή του είναι 1, που σημαίνει ότι η είσοδος **Sign_extend** θα περάσει στην ALU

▪ ALUOp: είναι 10, υποδεικνύοντας ότι η ALU θα εκτελέσει μια εντολή **R-type** εντολή, όπως ορίζεται στον αντίστοιχο πίνακα λειτουργιών

▪ Function_opcode: η τιμή είναι 100101, η οποία, όπως φαίνεται, αντιστοιχεί στη λογική πράξη **OR**

▪ PC_plus_4: ορίζεται χειροκίνητα ως 00010000

2. Έξοδοι:

- **ALU_Result:** το αποτέλεσμα της πράξης **OR** μεταξύ του **Read_data_1** και του **Read_data_2** είναι 00000000000000000000000000000101, δηλαδή 5

- **Add_Result:** υπολογίζεται με την πρόσθεση του **PC_plus_4** (00010000) και του **Sign_extend** (00000100), δίνοντας 00001000:
 - Παρατηρήσεις:
 - Τα δύο λιγότερο σημαντικά bits του **PC_plus_4** αγνοούνται (PC_plus_4(7 downto 2))
 - Ο υπολογισμός βασίζεται μόνο σε αυτά τα bits και στα 8 λιγότερο σημαντικά bits του **Sign_extend**
- **Zero:**
 - Η τιμή του είναι 1 αν το αποτέλεσμα του **ALU_Result** είναι 0
 - Εδώ, είναι 0, που είναι σωστό, καθώς το αποτέλεσμα είναι 5

Βήματα Υλοποίησης

- ✓ Ξεκινήστε ανοίγοντας το λογισμικό Quartus Prime
- ✓ Δημιουργήστε ένα νέο Project
- ✓ Ακολουθήστε προσεκτικά τις οδηγίες δημιουργίας Project, που βρίσκονται στη διαδρομή **eClass > Έγγραφα > Λογισμικά > Quartus Prime**
- ✓ Δημιουργήστε ένα νέο αρχείο VHDL (**New → VHDL File**)
- ✓ Γράψτε τον κώδικα και πατήστε το κουμπί play (compile) για να ελέγξετε την ορθότητά του
- ✓ Επιλέξτε **File → Create/Update → Create Symbol Files for Current File** για να μετατρέψετε τον κώδικα σε σύμβολο
- ✓ Ανοίξτε ένα νέο αρχείο Block Diagram (**File → New → Block Diagram/Schematic File**). Κάντε **δεξί κλικ → Insert → Symbol** → στο **Project** εισάγετε το σύμβολο και χρησιμοποιήστε το για την υλοποίηση του κυκλώματος
- ✓ Πατήστε το κουμπί play και ελέγξτε την ορθότητα της λειτουργίας του κυκλώματος.
- ✓ Ανοίξτε ένα νέο αρχείο Vector Waveform File (**New → Insert Node → Node Finder → List** → επιλέξτε τις εισόδους/εξόδους και πατήστε Ok
- ✓ Πατήστε **Simulation → Waveform Editor > Run Functional Simulation** για να εκτελέσετε την προσομοίωση και να παρακολουθήσετε τις κυματομορφές

Οδηγίες Υλοποίησης

- ✓ **Δημιουργία νέου project:** βεβαιωθείτε ότι ο φάκελος του project σας δεν θα βρίσκεται στην ίδια διαδρομή (path) του Quartus Prime (intel fpga)
- ✓ **Δημιουργία αρχείου VHDL:** το όνομα του αρχείου VHDL πρέπει να είναι ίδιο με το όνομα της οντότητας (entity) που περιέχεται στον κώδικα
- ✓ **Αποθήκευση Block Diagram και Vector Waveform File:** αποθηκεύστε αυτά τα αρχεία με το προτεινόμενο από το λογισμικό όνομα, ώστε να διασφαλιστεί η συμβατότητα
- ✓ **Εκτέλεση (compile) στοιχείων:** μεταβείτε στο **Project** → **Set as Top-level Entity** για να ορίσετε την κύρια οντότητα και στη συνέχεια play (compile)
- ✓ **Ρύθμιση εργαλείου προσομοίωσης:** μεταβείτε στο **Tools** → **Options** → **Eda Tool Options** και στο πεδίο **ModelSim**, καθορίστε τη διαδρομή "C:\intelFPGA_lite\20.1\modelsim_ase\win32aloem"
- ✓ **Ρύθμιση Simulation Settings:** ανοίξτε το menu **Simulation** → **Simulation Settings** και στην 5η γραμμή που εμφανίζεται η εντολή **vsim -novopt**, αφαιρέστε το **-novopt** για βελτιωμένη λειτουργία

Συγγραφή Εργασίας

Το αρχείο αποστολής της Εργασίας θα περιέχει ένα **zip** με:

- ✓ Τον φάκελο των αρχείων της εργασίας (βλ. επόμενες διαφάνειες)
- ✓ Αναφορά εργασίας (**Word - .docx**) η οποία θα τηρεί τα πρότυπα της Εργασίας που έχει αναρτηθεί στο **eClass** του Μαθήματος (*Εργαστήριο → Template Εργασιών → Υπόδειγμα Εργασίας - Ψηφιακά Ηλεκτρονικά*), δηλαδή θα περιέχει:
 - Εξώφυλλο
 - Ενότητες
 - Περιγραφή Εργασίας
 - Screenshot από τα κυκλώματα (με Print Screen ή Snipping Tool)
 - Τους υλοποιημένους κώδικες
 - Κυματομορφές που θα επαληθεύουν την ορθή λειτουργία του κυκλώματος και ανάλυση αυτών
 - Συμπεράσματα - Σχόλια – Πίνακες επαλήθευσης

Παράδοση Εργασίας (1)

Η παράδοση της Εργασίας θα πρέπει να γίνει στον ιστοχώρο της Ασύγχρονης Τηλεκπαίδευσης - eClass (<https://www.dit.uoi.gr/e-class/courses/201/>), μέχρι την καθορισμένη ημερομηνία.

Η εργασία που θα αποσταλεί θα πρέπει να περιλαμβάνει:

1. Αρχείο **Word (.docx)** με:

- ✓ Μια σελίδα με τα στοιχεία σας (Επώνυμο, Όνομα, ΑΜ, Email)
- ✓ Μια σελίδα με περιγραφή της λειτουργίας του κώδικα

2. Αρχεία κώδικα **VHDL (.vhd)**

- ✓ Ο κώδικας θα πρέπει να περιέχει σχόλια σε κάθε τμήμα του
- ✓ Στην αρχή κάθε κώδικα θα πρέπει να περιέχεται το πρότυπο δήλωσης δικαιωμάτων:

©Επώνυμο, Όνομα, Έτος
Με επιφύλαξη παντός δικαιώματος. All rights reserved.

Παράδοση Εργασίας (2)

Η παράδοση της Εργασίας θα πρέπει να γίνει στον ιστοχώρο της Ασύγχρονης Τηλεκπαίδευσης - eClass (<https://www.dit.uoi.gr/e-class/courses/201/>), μέχρι την καθορισμένη ημερομηνία.

Η εργασία που θα αποσταλεί θα πρέπει να περιλαμβάνει:

3. Τα αρχεία **Block Diagram (.bdf)** και **Waveform (.vwf)**
4. Το όνομα του αρχείου αποστολής (.docx ή .zip) θα είναι στη μορφής **Επώνυμο_AM_Εργασία_3.docx** (π.χ. Sakkas_00000_Ergasia_3.docx)
5. Στο τελικό αρχείο αποστολής (.zip) θα περιέχεται το αρχείο **.docx** συνοδευόμενο **ΜΟΝΟ** από τα αρχεία **.vhd** με τους κώδικες, με τα αντίστοιχα **.bdf** και **.vwf**